Project 2: Simplified TCP Protocol

Introduction

In this project, you will implement a simplified version of the TCP protocol on top of UDP. Your implementation will support: connection management, reliable data transfer, flow control and congestion control.

Be prepared: this is a single-person project and your skills will be exercised. So start early and feel more than welcome to ask questions. However, please note that the TAs are not allowed to debug your code for you during their office hours. They cannot touch your keyboard, and they will only spend a maximum of 10 minutes reading your code. This helps them to assist more students effectively. Therefore, it is recommended that you visit their office hours with specific questions, instead of vague statements like "my code doesn't work." For debugging, logging or tools like Wireshark[1] can be helpful. There are many ways to do this; be creative.

Starter Code

You are provided with some starter code in Python.

`grading.py`: these are variables that we will use to test your implementation, you do not need to make any changes here as we will be replacing it when running tests. But you can change some of the variables for your own testing. For example, you can change MSS to different values for testing reliability data transfer.

`transport.py`: it implements the core protocol logic for a simplified TCP protocol over UDP. This file provides a packet header definition, basic stop-and-wait style reliable delivery of a single segment at a time, and a dedicated thread (the backend) that handles both incoming packets (e.g., `ACKs`, data, `FINs`) and retransmissions. This design supports bidirectional communication, meaning each socket can send and receive data concurrently.

`client.py`: it contains the client-side application that uses the provided socket APIs defined in `transport.py`. It sends various types of data (e.g., file data and randomly generated data) to the server. It then waits to receive data from the server and prints out what is received.

[1] https://www.wireshark.org/

`server.py`: it contains the server-side application that uses the same socket APIs. It receives data from the client, prints the received data, and then sends its own data (for example, a file and some random data) back to the client.

Project Structure

This project is divided into **two checkpoints (two separate due dates)** to ease grading and help you focus on different components progressively.

Checkpoint 1 Tasks

- Connection management: implement connection establishment and termination using the simplified TCP FSM specified below.

- Flow control: implement a sliding window mechanism [1] that ensures the sender does not transmit more data than the receiver's advertised window. Enforce that the total amount of data buffered in the receive buffer does not exceed `MAX_NETWORK_BUFFER`.

- RTT Estimation: implement basic RTT estimation (using an EWMA) to adjust the retransmission timeout dynamically. You can implement the original algorithm specified in [2]. The α value can be any value between 0 and 1. You can compare the differences of different α.

Optional Tasks for Checkpoint 1

The following tasks are **optional** for 325 students but **REQUIRED** for 425 students. The 325 students who complete these tasks will receive extra credit.

- Selective ACK (SACK): implement SACK processing at the receiver so that non-contiguous data blocks are reported to the sender, allowing it to avoid unnecessary retransmissions.

- Duplicate ACK Retransmission (TCP Reno behavior): Detect triple duplicate ACKs and retransmit the missing segment without waiting for the timeout. The starter code only relies on timeouts to detect packet loss.

Checkpoint 2 Tasks

Once you have implemented the TCP basics, you can add a congestion control algorithm to handle potential network congestion.

- Update your code from Checkpoint 1 to add a new parameter, the congestion window: `cwnd`. The size of your sending window should now be the minimum of `cwnd` and the advertised window. You should additionally maintain that the total amount of data buffered for the application (unread data, both ordered and unordered bytes) should be less than `MAX_NETWORK_BUFFER`, a parameter defined in the starter code.

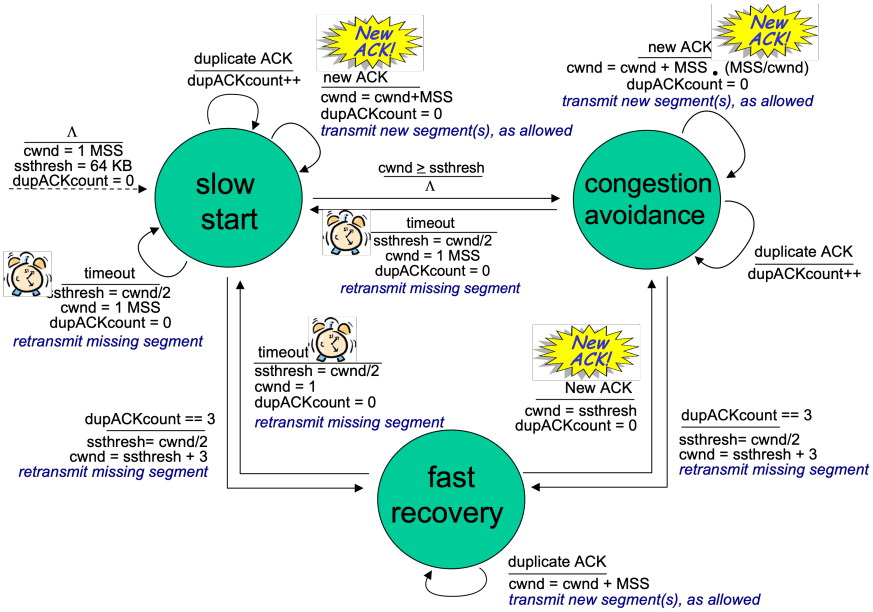
- Congestion Control - Implement the slow start and congestion avoidance features of the state machine. This should make your implementation like TCP Tahoe. Initially, set `cwnd` to `WINDOW_INITIAL_WINDOW_SIZE` and `ssthresh` to `WINDOW_INITIAL_SSTHRESH`.

Optional Tasks for Checkpoint 2

The following tasks are **optional** for 325 students but **REQUIRED** for 425 students. The 325 students who complete these tasks will receive extra credit.

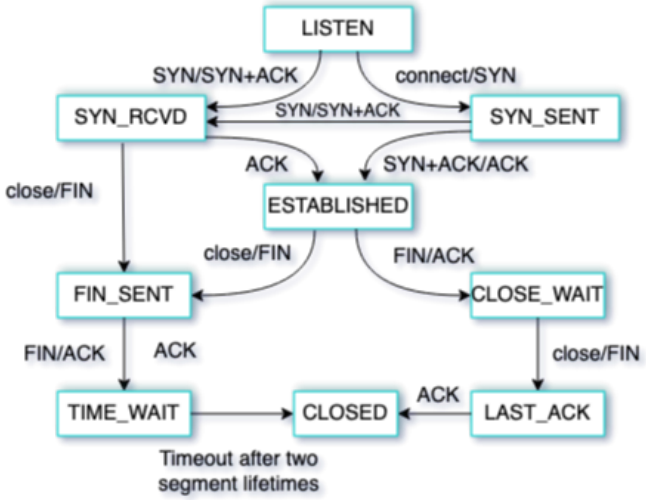
- You can also implement fast recovery, this should move your implementation from Tahoe to Reno.

Below is a diagram showing the FSM for TCP congestion control, same as the one we discussed in class.



Simplified TCP Connection Management FSM

Your implementation will follow a simplified version of TCP's state machine. (Note: Connection management is not provided in the starter code and should be implemented by you.) The simplified FSM looks like below.



The `FIN_WAIT_1`, `FIN_WAIT_2`, and `CLOSING` states have been eliminated from this FSM. Each state transition in this FSM denotes the events that trigger the transition and the actions taken upon receiving the event. For example, the state transition between `FIN_SENT` and `TIME_WAIT` can be triggered by the receipt of a `FIN` packet or an `ACK` packet (to the previously sent `FIN`). If a `FIN` is received, an `ACK` will be sent to acknowledge the receipt of this `FIN`; if an `ACK` is received, no further actions need to be taken.

Note that in this project, we don't expect you to piggyback `ACK` within `FIN` or other data packets to simplify implementations. In other words, `ACK` will be in its own packet except for the `SYN-ACK` packet.

Packet Header Format

Your implemented protocol should use a custom packet header for the simplified TCP over UDP. The header should include the following fields:

Field Name	Size (bits)	Description
Sequence Number	32	Sequence number of the first byte in the payload.

Acknowledgment Number	32	Next expected byte from the sender (cumulative acknowledgment).
Flags	8	Control flags (e.g., `SYN`, `ACK`, `FIN`).
Advertised Window	16	Receiver's available buffer size for flow control.
SACK Block (Optional)	64	A single Selective Acknowledgment block (32 bits for left edge and 32 bits for right edge).

All fields are in network byte order. For this project, you may assume a fixed header length. For 325 students, if you don't implement SACK, you can ignore the SACK Block field in the header. Otherwise, you may assume that at most one SACK block is allowed.

Testing Your Implementation Under Various Network Conditions

To test the functionality of your implementation, we provide a simple client and server that exchange data using APIs, such as `send()` and `recv()`.

- The client program uses your TCP implementation to send data (e.g., files and random data) to the server.
- The server program uses your TCP implementation to receive data from the client and then send some other data back.

To thoroughly test your TCP implementation—including its connection management, flow control, congestion control, and reliability features—you should evaluate its behavior under different network conditions. You can use the Linux `tc` (traffic control) command with the `netem` (network emulator) module to simulate these conditions.

Using `tc` to Simulate Packet Loss and Delay

For example, to simulate a network scenario with **100 ms delay** and **10% packet loss** on the loopback interface (`lo`), run the following command in your terminal:

```
sudo tc qdisc add dev lo root netem delay 100ms loss 10%
```

This command sets up a queueing discipline (qdisc) on the `lo` interface that:

- **Adds a 100 ms delay** to every packet transmitted on the interface.

- **Drops 10% of the packets** at random, simulating a lossy network.

Restoring Normal Network Conditions

When you're finished testing, remove the network emulation with:

```
sudo tc qdisc del dev lo root netem
```

This command deletes the qdisc previously added, returning the network interface to its normal behavior.

Testing Guidelines:

Test your connection management and reliable data transfer by running both server and client on the same host with various network conditions.

Monitor the logs (using logging statements or Wireshark) to observe how your protocol handles delays and losses:

- Confirm that timeouts trigger retransmissions.
- Verify that the congestion window (`cwnd`) is adjusted according to slow start and congestion avoidance rules.
- Check that your flow control prevents the receiver's buffer from exceeding `MAX_NETWORK_BUFFER`.

Experiment with different settings

- Vary the delay (e.g., 50ms, 200ms) and packet loss percentage (e.g., 5%, 20%) to observe the protocol's behavior under various conditions.
- Use these experiments to generate graphs for your submission, showing performance metrics such as RTT, throughput, and the evolution of `cwnd`.

Submission Instructions

Submit the following files as a single ZIP archive:

1. **transport.py**: Your implementation of the simplified TCP protocol.

2. **README**: A short description of your implementation, any assumptions made, and instructions to run your code.

REMINDERS

- Submit your code to Canvas in a zip file
- If your code does not **run**, you will not get credits.
- Document your code (by inserting comments in your code)
- **Checkpoint 1 DUE: 11:59 pm, Friday, Mar 26th.**
- **Checkpoint 2 DUE: 11:59 pm, Friday, April 9th.**

References

[1] TCP Sliding Window:
https://book.systemsapproach.org/direct/reliable.html#sliding-window
https://book.systemsapproach.org/e2e/tcp.html#sliding-window-revisited

[2] Adaptive Retransmission:
https://book.systemsapproach.org/e2e/tcp.html#adaptive-retransmission.